

Transformer Architecture — Complete Interview & Learning Notes

1. What is a Transformer?

A Transformer is a deep learning architecture designed to understand relationships between different parts of a sequence using a mechanism called Attention, especially Self-Attention.

It was introduced in the 2017 research paper:

"Attention Is All You Need" — Vaswani et al.

Transformers are the foundation of modern AI models such as BERT, GPT, T5, LLaMA, and many other Large Language Models (LLMs).

Simple Definition

A Transformer is a neural network architecture that processes sequences by learning which words or tokens are important to each other using self-attention.

2. Why Was the Transformer Needed?

Before Transformers, sequence-based tasks were mainly handled using:

RNN (Recurrent Neural Network)

LSTM (Long Short-Term Memory)

GRU (Gated Recurrent Unit)

Problem with RNN and LSTM

RNNs process words one by one.

Example:

I → love → learning → Artificial → Intelligence

This creates two major problems:

1. Sequential Processing

The model cannot efficiently process all words in parallel.

2. Long-Term Dependencies

It can struggle to remember relationships between words that are far apart.

Example:

The student who studied AI and machine learning for several years was successful.

The model needs to understand the relationship between:

student → was successful

even though many words appear between them.

Transformer Solution

Transformers use Self-Attention, allowing every word to look at every other word simultaneously.

Every Token ↔ Every Other Token

This makes Transformers:

Highly parallelizable

Better at understanding context

Effective for long-range relationships

Suitable for large-scale training

3. High-Level Transformer Architecture

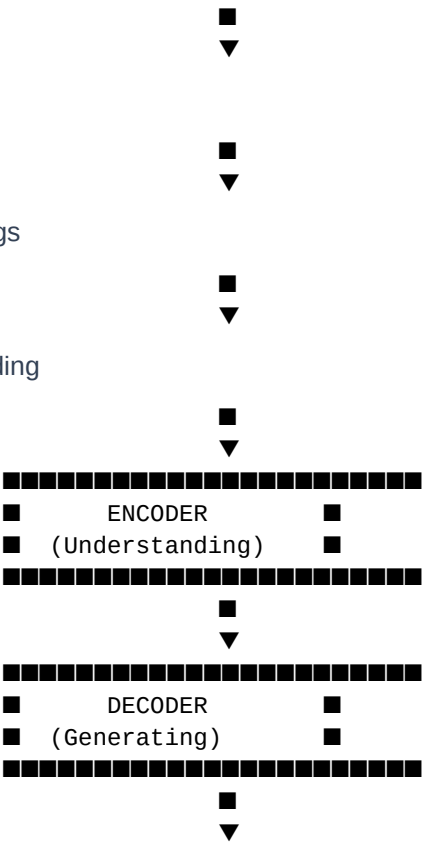
The original Transformer consists of two main parts:

INPUT TEXT

Tokenization

Input Embeddings

Positional Encoding



Output Text

Important Modern Architecture Types

Encoder Only → BERT
Decoder Only → GPT, LLaMA
Encoder-Decoder → T5, Original Transformer

4. Step 1: Tokenization

Computers do not directly understand words. Text is first divided into smaller units called tokens.

Example:

"I love AI"

Can become:

["I", "love", "AI"]

Then tokens are converted into numerical IDs:

"I" → 101
"love" → 205
"AI" → 450

These token IDs are the input to the model.

5. Step 2: Token Embeddings

A token ID alone has no semantic meaning.

Therefore, each token is converted into a dense numerical vector called an embedding.

Token ID



Embedding Layer



Dense Vector

Example:

"king" → [0.12, 0.87, -0.34, ...]
"queen" → [0.15, 0.82, -0.30, ...]

Embeddings allow the model to represent semantic relationships mathematically.

6. Why Do Transformers Need Positional Encoding?

Self-Attention processes all tokens simultaneously.

Because of this, the Transformer does not automatically know the order of words.

Consider:

Dog bites man

and:

Man bites dog

The same words have completely different meanings.

Therefore, positional information is added to token embeddings.

Final Input Representation =

Token Embedding + Positional Encoding

Example:

Position 1 → I
Position 2 → love
Position 3 → AI

This allows the model to understand word order and position.

7. The Most Important Concept: Self-Attention

What is Attention?

Attention answers the question:

When understanding a word, which other words should the model focus on?

Consider:

"The animal didn't cross the street because it was too tired."

To understand what "it" refers to, the model should pay attention to:

animal

Self-Attention allows each token to examine all other tokens and determine their importance.

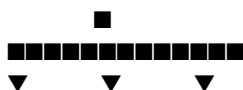
8. Query, Key, and Value (Q, K, V)

This is one of the most important Transformer concepts for interviews.

Each input embedding is transformed into three vectors:

Query (Q) → What am I looking for?
Key (K) → What information do I contain?
Value (V) → What information should I provide?

Input Embedding



Q K V

Simple Analogy: Search Engine

Imagine you search Google.

Query → Your search request
Key → Information used to match results
Value → Actual information returned

In Self-Attention, each token creates its own Q, K, and V vectors.

9. How Self-Attention Works

The Self-Attention process happens in three main steps.

Step 1: Calculate Similarity Between Query and Key

The Query of one token is compared with the Keys of all tokens.

Attention Score = $Q \times K^T$

Step 2: Scale and Apply Softmax

The scores are divided by the square root of the Key dimension and passed through Softmax.

Attention Weights =

$$\text{Softmax}(QK^T / \sqrt{d_K})$$

Softmax converts the scores into probabilities.

Example:

Token A → 0.10
Token B → 0.70
Token C → 0.20

This means Token B receives the most attention.

Step 3: Multiply Attention Weights with Values

The attention weights are used to combine the Value vectors.

Complete Formula

Attention(Q, K, V) =

$$\text{Softmax}(QK^T / \sqrt{d_K}) \times V$$

In Simple Words

Each word decides which other words are important and combines information from them to create a context-aware representation.

10. Example of Self-Attention

Sentence:

"The cat sat on the mat"

When processing the word "sat", the model may assign different attention scores:

The → 0.05
cat → 0.60
sat → 0.10
on → 0.05
the → 0.05
mat → 0.15

The model learns that "cat" is highly relevant to understanding "sat".

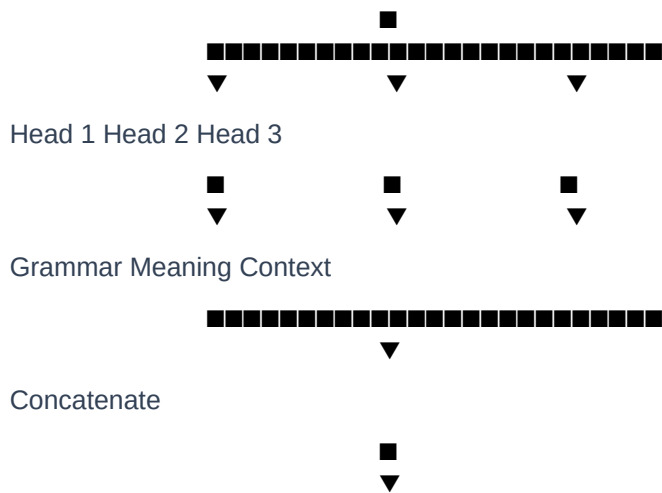
These values are learned during training, not manually programmed.

11. Multi-Head Attention

A single attention mechanism may focus on only one type of relationship.

Therefore, Transformers use multiple attention heads.

Input



Final Output

Different attention heads can learn different relationships, such as:

Grammar relationships

Subject–verb relationships

Long-distance dependencies

Semantic meaning

Positional relationships

Interview Definition

Multi-Head Attention allows the model to attend to different parts and different representation subspaces of the input simultaneously.

12. Transformer Encoder

The Encoder is responsible for understanding the input.

The original Transformer encoder contains multiple identical layers.

Each Encoder layer contains:

Input



Multi-Head Self-Attention



Add & Layer Normalization



Feed Forward Neural Network



Add & Layer Normalization



Output

Encoder Components

A. Multi-Head Self-Attention

Every token can attend to every other token.

B. Add & Norm

The Transformer uses:

Residual Connections

Layer Normalization

Conceptually:

$$\text{Output} = \text{LayerNorm}(\text{Input} + \text{Sublayer}(\text{Input}))$$

Residual connections help information and gradients flow through deep networks.

Layer Normalization helps stabilize training.

C. Feed Forward Neural Network (FFN)

After attention, every token passes through a neural network independently.

Conceptually:

$$\text{FFN}(x) = \text{Activation}(xW_1 + b_1)W_2 + b_2$$

The FFN helps the model learn more complex transformations.

13. Transformer Decoder

The Decoder is responsible for generating output tokens.

Example:

Input: Translate English to French: "Hello"

Output: Bonjour

A Decoder layer typically contains:

Masked Multi-Head Attention



Add & Layer Normalization



Cross Attention



Add & Layer Normalization



Feed Forward Network



Add & Layer Normalization

14. What Is Masked Self-Attention?

During text generation, the model should not see future words.

Suppose the sentence is:

I love Artificial Intelligence

When predicting:

love

the model should not look ahead at:

Artificial Intelligence

Therefore, a causal mask hides future tokens.

Current Token → Can see previous tokens
Current Token → Cannot see future tokens

This is essential for autoregressive models such as GPT.

15. What Is Cross-Attention?

Cross-Attention is mainly used in Encoder–Decoder architectures.

The Decoder uses:

Query → From Decoder
Key → From Encoder
Value → From Encoder

This allows the Decoder to focus on relevant information from the input while generating output.

Example:

English Input

↓

Encoder understands meaning

↓

Decoder attends to Encoder output

↓

French Output

16. Transformer Architecture Types

Encoder-Only Models

Example: BERT

Input → Encoder → Understanding / Classification

Used for:

Text classification

Sentiment analysis

Named Entity Recognition

Question answering

Key Characteristic

BERT can generally use bidirectional context, meaning tokens can learn from both left and right context during its encoder processing.

Decoder-Only Models

Examples: GPT and LLaMA

Previous Tokens → Decoder → Next Token Prediction

Used for:

Text generation

Chatbots

Code generation

LLM applications

Key Characteristic

Uses causal or masked self-attention to predict the next token.

Encoder-Decoder Models

Example: T5

Input → Encoder → Decoder → Output

Used for:

Translation

Summarization

Text-to-text tasks

17. How GPT Generates Text

Suppose the prompt is:

"Artificial Intelligence is"

The model predicts probabilities for the next token:

transforming	→ 40%
a	→ 25%
changing	→ 15%
powerful	→ 10%

...

One token is selected:

Artificial Intelligence is transforming

Then the process repeats:

Artificial Intelligence is transforming the

Artificial Intelligence is transforming the world

This is called autoregressive generation.

Previous Tokens → Predict Next Token → Repeat

18. Training vs Inference

Training

During training, the model learns from a large dataset.

Example:

Input: I love machine

Target: learning

The model:

Makes a prediction

Calculates loss

Uses backpropagation

Updates model weights

This process is repeated billions of times.

Inference

During inference, the trained model generates output.

User Prompt

↓

Tokenizer

↓

Transformer Model

↓

Next Token Prediction

↓

Generated Response

No model weights are updated during normal inference.

19. Why Are Transformers So Powerful?

Parallel Processing

Unlike RNNs, Transformers can process training sequences in parallel.

Context Understanding

Self-Attention captures relationships between tokens.

Scalability

Transformers can be scaled with:

More parameters

More data

More compute

Transfer Learning

A pretrained Transformer can be adapted for different tasks.

20. Limitations of Transformers

Transformers are powerful, but they have limitations.

1. Computational Cost

Standard Self-Attention compares tokens with each other, leading to approximately:

$O(n^2)$

complexity with respect to sequence length.

Longer contexts can therefore become computationally expensive.

2. Large Data Requirements

Large models typically require enormous datasets.

3. High Hardware Requirements

Training large Transformers requires significant GPU/TPU resources.

4. Hallucination

LLMs can generate information that sounds convincing but is incorrect.

5. Lack of True Understanding

Transformers learn statistical patterns and representations from data. High-quality language behavior should not automatically be interpreted as human-like understanding or consciousness.

21. Complete Transformer Flow

TEXT INPUT



TOKENIZATION



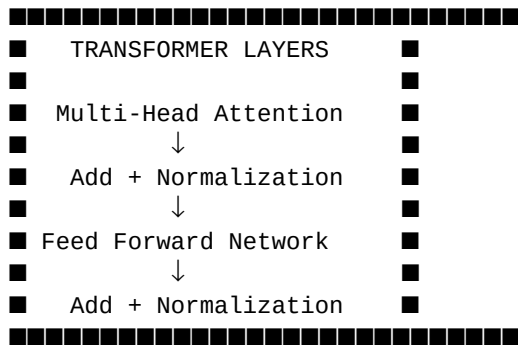
TOKEN IDs



TOKEN EMBEDDINGS



POSITIONAL INFORMATION



CONTEXTUAL REPRESENTATION



OUTPUT PREDICTION

22. Transformer Interview Questions You Must Know

Basic Level

What is a Transformer?

Why was the Transformer introduced?

What problem does Self-Attention solve?

What are Token Embeddings?

Why is Positional Encoding needed?

Intermediate Level

Explain Query, Key, and Value.

Explain the Self-Attention formula.

What is Multi-Head Attention?

What are Residual Connections?

What is Layer Normalization?

What is the Feed Forward Network in a Transformer?

What is the difference between Encoder and Decoder?

Advanced Level

What is Masked Self-Attention?

What is Cross-Attention?

Why does Self-Attention have quadratic complexity?

What is the difference between BERT and GPT?

Explain autoregressive generation.

How does a Transformer handle long-range dependencies?

What happens during Transformer training?

What are the limitations of Transformers?

23. The Most Important Interview Answer

Explain Transformer Architecture in an Interview

A Transformer is a deep learning architecture that processes sequential data primarily using the attention mechanism instead of recurrence. The input text is first tokenized and converted into embeddings, with positional information added so the model can represent token order. The core component is multi-head self-attention, where each token creates Query, Key, and Value representations to determine which other tokens are relevant. The attention output is combined with residual connections, layer normalization, and feed-forward neural networks across multiple layers. Depending on the architecture, Transformers can use an encoder for understanding, a decoder for autoregressive generation, or both for sequence-to-sequence tasks. Modern models such as BERT, GPT, and T5 are based on different variants of this architecture.

■ One-Line Revision

Transformer = Embeddings + Position Information + Attention + Feed Forward Networks

■ The Core Concept to Remember

Self-Attention allows every token to dynamically gather relevant information from other tokens and build a context-aware representation.

This is the foundation you should understand before moving to BERT, GPT, LLM architecture, RAG, fine-tuning, or Generative AI interviews.